Basic (design on paper before touching code) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The three designs to sketch, using this document's Core Concepts table as your reference for what each one actually means:

- **Single agent** — one agent, one system prompt, with both a "research" tool and a "write" ability. No handoffs at all.
- **Sequential** — two agents (or two clearly separated steps), research always running first, its output feeding directly into the write step, in a fixed order.
- **Supervisor** — a router agent that decides which specialist to call, in what order, based on the task — even though for this simple task the order is probably still research-then-write, the supervisor decides that dynamically rather than it being hardcoded.

For each, write one paragraph describing the flow, then one line predicting: cost (roughly how many LLM calls), speed (roughly how much wall-clock delay), and reliability (what's most likely to go wrong).

The exact pieces:

- **Cost** — single agent: 1-2 calls (it might call its research tool internally, then answer). Sequential: 2 calls minimum, one per stage. Supervisor: 2 calls for the work, plus 1+ extra calls for the routing decision itself — the supervisor's own reasoning about who to call isn't free.
- **Speed** — single agent and sequential both run everything one after another (no parallelism here, since write genuinely needs research's output first). Supervisor adds the routing decision's own delay on top of sequential's delay.
- **Reliability** — single agent: fewer moving parts, but a single system prompt trying to be good at two different jobs can do both a little worse. Sequential: very predictable, since each stage has

one clear job — but a bad research result guarantees a bad write result, with nothing to catch it. Supervisor: adds a new failure mode entirely — the supervisor routing to the wrong specialist, or looping between them unnecessarily.

Write your three paragraphs and three one-line predictions, then go one step further before Hint 2: a first-pass guess usually undercounts two things — hidden costs, and how a design fails, not just whether it "works."

On hidden costs — the supervisor's routing decision is easy to write off as "basically free," since it's not doing the actual research or writing work. It isn't free: it's a real LLM call, reasoning over the current state, and for a task this small it can easily cost as much as one of the two real work stages. Write down, honestly, whether your supervisor prediction accounted for that, or quietly assumed routing was instant and free.

On failure modes — don't just ask "does this design work?", ask "what does it do when something goes wrong partway through?" A single agent that mishandles its research tool call still produces *something*, confidently, with nothing external to catch it. Sequential's research stage failing propagates straight through to write, guaranteed, since nothing checks the handoff. Supervisor adds a failure mode neither of the other two has at all: routing to the wrong specialist, or calling the same one twice because its own state-tracking is off.

The real design question isn't just "which design is fastest on paper" — it's "which design's most likely failure would you actually notice, and which would quietly produce a confident-sounding wrong answer?"

The extra pieces needed:

- A one-line "if this goes wrong, here's what happens" note added under each of your three paragraphs, not just the happy-path flow.
- An honest accounting of the supervisor's routing call as a real, separately-counted cost — not folded silently into "2 calls for the work."
- The out-loud defense this document's "How to Practice" section asks for: could you explain to a skeptical senior engineer, in one sentence, why the task you picked would or wouldn't justify 3 agents over 1?

Sketch this hardened version yourself before checking Hint 2.

Hint 2 — The plan, and almost the whole thing

```
task: "research the top 3 causes of X, then write a 3-sentence summary"

design 1 -- single agent
    flow: one agent gets the task, calls a research tool as needed, then writes
    the summary itself in the same turn
```

```
    cost: ~1-2 LLM calls
    speed: fastest -- no handoff overhead
    reliability risk: the single system prompt has to be good at both research
    judgment and writing style at once

design 2 -- sequential
    flow: research_agent(topic) -> summary; write_agent(summary) -> final text
    cost: ~2 LLM calls
    speed: same order as design 1, roughly, plus a small handoff cost
    reliability risk: no reliability improvement over research quality itself --
    if research is bad, the write step faithfully writes a bad summary

design 3 -- supervisor
    flow: supervisor reads the task, decides to call research_agent,
    gets result, decides to call write_agent, gets result, returns
    cost: ~3+ LLM calls (2 for the work, 1+ for routing)
    speed: slowest of the three -- routing decisions add real delay
    reliability risk: everything sequential has, plus the supervisor could route
    to the wrong specialist or loop unnecessarily
```

Fill in your own chosen task the same way — one paragraph and one cost/speed/reliability line per design — then go one step further, the same way Hint 1 did: add the failure-mode line and the honestly-counted routing cost.

Here's almost the whole write-up for a different task ("plan a trip"), with the failure-mode line Hint 1 asked for — fill in your own task the same way, don't copy this one directly:

```
task: "plan a 3-day trip: find flights, then find a hotel near the
arrival airport"

single agent (one agent, two tools):
    cost: ~1-2 calls -- the agent can call both tools in one turn if
    it reasons well
    speed: fastest -- no agent-to-agent handoff delay
    reliability risk: it might search for a hotel before it has flight
    dates, if its own internal reasoning doesn't enforce the order --
    nothing external enforces it
    if it goes wrong: a confident-sounding hotel suggestion with no real
    flight dates behind it -- nothing flags this, since the same agent
    "trusts itself"

sequential (flight_agent -> hotel_agent, fixed order):
    cost: ~2 calls, one per stage
    speed: medium -- hotel search genuinely can't start until flight
    dates are known, so there's no real parallelism to lose here anyway
    reliability risk: low for ordering (it's enforced by code, not the
```

```
    model's judgment), but a flight_agent mistake still propagates
    straight through
    if it goes wrong: a bad flight pick guarantees a hotel search built
    on it -- the failure is very visible in the flight step, at least,
    since nothing hides it

supervisor (router decides who runs next):
    cost: highest -- the routing decision itself is 1+ extra calls,
    real and separately counted, not folded into "2 calls for the work"
    speed: slowest -- same work as sequential, plus routing overhead
    reliability risk: everything sequential has, plus a new one: the
    router could send the task to hotel_agent before flight_agent has run
    if it goes wrong: the router silently loops flight_agent twice, or
    sends hotel_agent an empty flight-dates field it doesn't know how
    to handle
```

What's missing: your own task, filled in the same way including the "if it goes wrong" line, plus the follow-up commitment to actually build the sequential and supervisor versions in the next two exercises and compare against these guesses. Do that, then check the Solution.

Full solution: Show me the solution